

BARANGAY MINANTE 1 INTEGRATED MANAGEMENT SYSTEM

Complete System Reviewer, Technical Documentation,
and Code Explanation

RESEARCH • SYSTEM DEFENSE • TECHNICAL REFERENCE • MAINTENANCE

Development of an Integrated Barangay Permit, Event Approval, Blotter Management System with
Notification and Scheduling Features for Barangay Minante 1, Cauayan City

Barangay Minante 1, Cauayan City, Isabela

Document Version 2.1 • September 9, 2026

Prepared from direct inspection of the current working source code and database schema.

Table of Contents

Table of Contents	2
1. Executive Summary	6
Level 1 — Simple explanation	6
Level 2 — Technical explanation	6
2. The Barangay Problems Addressed	7
Important research qualification	7
Problem 1: Fragmented and difficult-to-find records	7
Problem 2: Repeated visits just to ask for status	7
Problem 3: No immediate proof that a request was received	7
Problem 4: Unclear responsibility between Staff and Administrator	7
Problem 5: Inconsistent transaction steps	7
Problem 6: Scheduling conflicts and disconnected appointment notes	7
Problem 7: Delayed or inconsistent communication	8
Problem 8: Limited management visibility	8
Problem 9: Weak accountability for important actions	8
Problem 10: Account and password security	8
Problem-to-solution summary	9
3. Proposed Solutions and User Benefits	10
Benefits for Residents	10
Benefits for Barangay Staff	10
Benefits for the Administrator	10
Organizational benefits	10
Benefits not yet proven	11
4. System Scope and Implementation Status	12
5. Input-Process-Output Model	13
Level 1 — Simple explanation	13
Inputs	13
Processes	13
Outputs	13
6. System Architecture and Data Flow	14
Architecture	14
Level 1 Data Flow Diagram explanation	14
7. Users, Roles, and Use-Case Flow	15
Resident	15
Barangay Staff	15
Administrator	15

Actual permission matrix	15
Use-case sequence	15
8. Authentication, Registration, and Password Recovery	16
Login	16
Registration	16
Authentication versus authorization	16
Forgot Password	16
9. Role-Specific Dashboards	17
Administrator Dashboard	17
Staff Dashboard	17
Resident Dashboard	17
10. Module 1.0 — User Management	18
Simple explanation	18
Technical explanation	18
Current limitations	18
11. Module 2.0 — Permit Processing	19
Workflow	19
Stored information	19
Submission and review	19
Approved document	19
12. Module 3.0 — Event Approval Management	20
Workflow	20
Stored fields	20
Actual current form behavior	20
13. Module 4.0 — Blotter Management	21
Simple explanation	21
Workflow	21
Data and privacy	21
Partial fields	21
14. Module 5.0 — Scheduling and Notifications	22
Scheduling	22
Notifications	22
Announcements	22
15. Submission Receipts and Approved Documents	23
16. Module 6.0 — Reports and Monitoring	24
17. Module 7.0 — Activity Logging	25

Simple explanation	25
Technical explanation	25
18. SQLite and Complete Database Architecture	26
SQLite in simple words	26
Connection architecture	26
SQLAlchemy versus raw SQL	26
Complete table reference	26
Primary and foreign keys	27
Simplified relationship diagram	27
Constraints and indexes	27
Render persistence boundary	27
19. CRUD, Transactions, ORM, and Reference Numbers	28
CRUD	28
Commit and rollback	28
References	28
20. File Uploads and Storage	29
21. Role-Based Access Control and Status Transitions	30
RBAC analogy	30
Enforcement layers	30
22. Frontend Architecture and User Experience	31
23. Programming Languages, Frameworks, and Libraries	32
24. Project Structure	33
25. Important Code Explanations	34
Database URL	34
SQLite connection protection	34
Password protection	34
Reference generation	34
Ownership	34
Status validation	34
Audit in the same transaction	34
Secure reset token storage	35
26. Route and Request Flow	36
27. End-to-End Use Cases	37
Example 1 — Permit	37
Example 2 — Event	37
Example 3 — Blotter	37

28. Security and Error Handling	38
Implemented protections	38
HTTP errors	38
Production hardening still required	38
29. From Click to Database: What Happens Behind the Screen	39
30. Feature-to-Code-to-Database Matrix	40
31. Strengths, Limitations, and Future Improvements	41
Confirmed strengths	41
Confirmed limitations	41
Recommended future work	41
32. Common Research and Defense Questions	43
33. Glossary	45
34. Quick Reviewer / Cheat Sheet	46
35. Verification Checklist	47
Final Summary	48

1. Executive Summary

Level 1 — Simple explanation

The Barangay Minante 1 Integrated Management System is a digital barangay office assistant. Residents can create an account, submit a permit application, request event approval, file a blotter complaint, request an appointment, follow a transaction, read announcements and notifications, and download proof that a request was received.

Barangay Staff use the system as a front-desk workspace. They review requests, move valid applications through the proper steps, process blotter cases, and manage schedules. The Administrator controls users and configuration, makes final permit and event decisions, monitors system activity, and generates reports.

Instead of relying only on paper folders, verbal follow-ups, and disconnected lists, the application puts authorized information in one SQLite database. A reference number acts like a tracking number. Statuses show where a request is in the process. Activity logs act like a digital security notebook.

Level 2 — Technical explanation

The current implementation is a server-rendered Flask application. `app.py` defines routes and business operations; `models.py` defines 13 SQLAlchemy models; `permissions.py` centralizes role-based permissions; `workflows.py` validates state transitions; `services/notify.py` creates notification records; and `services/pdfgen.py` generates PDF acknowledgments, approved permit documents, and reports. Jinja2 templates render the interface, while `static/css/style.css` supplies the responsive visual system.

Persistence uses Flask-SQLAlchemy with Python's built-in SQLite driver and an absolute database path assembled from environment variables in `config.py`. Locally the file is `instance/minante.db`; on Render it must be `/var/data/minante.db` on an attached Persistent Disk. Operational modules query real tables rather than dashboard sample arrays. Passwords are hashed by Werkzeug, reset tokens are stored only as SHA-256 hashes, mutating requests require a CSRF token, and Residents are filtered to their own records.

The application implements the seven research processes: User Management; Permit Processing; Event Approval Management; Blotter Management; Scheduling and Notification Management; Reports and Monitoring; and Activity Logging.

2. The Barangay Problems Addressed

Important research qualification

The source code proves what the system is designed to solve; it does not by itself prove how often each problem occurs in Barangay Minante 1. Claims about actual frequency, resident satisfaction, processing-time reduction, or paper-cost reduction require interviews, surveys, observation, or before-and-after measurements. The items below should therefore be described as identified operational problems addressed by the project, unless supported by separate field-study evidence.

Problem 1: Fragmented and difficult-to-find records

Simple explanation: Paper forms and separate files can be slow to search. A resident may have one permit paper, another appointment note, and a separate blotter record.

System response: The application stores users, permits, events, blotters, schedules, notifications, reports, and activity logs in related SQLite tables.

Benefit: Authorized personnel can search and open consistent digital records from one system. Residents see their own history without relying only on physical copies.

Problem 2: Repeated visits just to ask for status

Simple explanation: A resident may travel to the barangay hall only to ask, "Is my request ready?"

System response: Each submission receives a unique reference number and a visible workflow status. Residents have a transaction-status page, dashboard summaries, and notifications.

Benefit: Residents can check progress online, while Staff spend less time answering repetitive status inquiries.

Problem 3: No immediate proof that a request was received

Simple explanation: Without a receipt, a resident may be unsure whether a form was actually recorded.

System response: After a successful Permit, Event, or Blotter commit, the system opens a success page and allows an acknowledgment PDF to be downloaded again.

Benefit: The resident receives a date, time, reference number, summary, processing guidance, and proof of submission. The receipt clearly states that it is not final approval.

Problem 4: Unclear responsibility between Staff and Administrator

Simple explanation: If everyone can approve everything, mistakes and unauthorized decisions become more likely.

System response: Staff review and endorse Permit and Event records. The Administrator makes final approval or rejection decisions. Central permission and transition rules enforce this separation on the server.

Benefit: Responsibilities are easier to explain, decisions are more accountable, and Staff cannot perform restricted final decisions.

Problem 5: Inconsistent transaction steps

Simple explanation: A request should not jump from "new" directly to "finished" without review.

System response: `workflows.py` defines allowed status changes for each role and module. Invalid transitions produce a 400 response.

Benefit: Permit, Event, Blotter, and Schedule records follow predictable procedures.

Problem 6: Scheduling conflicts and disconnected appointment notes

Simple explanation: Appointment details written on separate paper can be forgotten or disconnected from the request they belong to.

System response: A first-class `schedules` table stores a reference, related transaction type and ID, requester, date/time, purpose, status, and confirming user.

Benefit: Residents and personnel can view appointments together with their related services. Schedules can be confirmed, rescheduled, completed, or cancelled through controlled transitions.

Problem 7: Delayed or inconsistent communication

Simple explanation: Residents need to know when a status or schedule changes.

System response: The system creates in-app, email, and SMS notification records when submissions and status changes occur.

Benefit: In-app communication is recorded and visible. The data model is ready for multiple channels.

Current limitation: Email and SMS delivery are simulated. A record can be created for these channels, but no external gateway actually sends the message yet.

Problem 8: Limited management visibility

Simple explanation: Leaders need totals to understand the workload.

System response: Role-specific dashboards calculate counts directly from SQLite. The Administrator Reports page groups Permit, Event, Blotter, Schedule, and Notification records and can export CSV or PDF summaries.

Benefit: The Administrator can see workload distribution and records awaiting decisions without manually counting papers.

Problem 9: Weak accountability for important actions

Simple explanation: It should be possible to know who changed a record and when.

System response: `log_activity()` stores the user, role, action, resource, reference, details, IP address, user agent, and timestamp.

Benefit: Administrators receive an audit trail for security review, troubleshooting, and accountability.

Problem 10: Account and password security

Simple explanation: Plain passwords and reusable reset links are unsafe.

System response: Passwords use one-way Werkzeug hashes. Reset links use random tokens whose SHA-256 hashes are stored, expire after 30 minutes by default, and become unusable after a successful reset.

Benefit: Database access does not reveal users' original passwords, and recovery links have limited lifetime and reuse.

Problem-to-solution summary

Barangay service problem	Implemented system solution	Main beneficiaries
Scattered records	Central SQLite tables and relationships	Staff, Administrator, Residents
Repeated status visits	Reference numbers, status page, notifications	Residents, Staff
No submission proof	Downloadable acknowledgment receipt	Residents
Unclear decision authority	RBAC and role-aware workflows	Staff, Administrator
Inconsistent processing	Validated transition maps	All roles
Disconnected appointments	Dedicated Schedule records	Residents, Staff
Communication gaps	Notification history and announcements	All roles
Slow manual counting	Live dashboard counts and exports	Administrator
Weak accountability	Activity log	Administrator, organization
Unsafe credentials	Password hashing and expiring reset tokens	All users

3. Proposed Solutions and User Benefits

Benefits for Residents

- Submit Permit, Event, and Blotter requests from one account.
- Receive a tracking reference and downloadable acknowledgment.
- View only their own transaction history and appointments.
- Read status updates, announcements, and in-app notifications.
- Update their profile and password without asking Staff to edit the database.
- Reduce uncertainty and unnecessary follow-up visits.

Benefits for Barangay Staff

- Use a front-desk queue for Pending and Under Review requests.
- Review and endorse Permit and Event requests without receiving final-decision authority.
- Process Blotter cases through intake, investigation, and hearing stages.
- Create and manage operational schedules.
- Read the active user list without receiving Administrator-level user controls.
- Work from consistent statuses and references instead of informal notes.

Benefits for the Administrator

- Control user roles and activation status.
- Preserve a permanent Administrator account during initialization and migration.
- Make final Permit and Event decisions.
- Manage announcements, service catalogs, and receipt processing guidance.
- Monitor live counts and export reports.
- Inspect up to 500 recent audit events in the Activity Logs page.

Organizational benefits

- Centralized and persistent data
- More consistent workflows
- Clearer separation of duties
- Traceable actions
- Faster information retrieval
- Reduced duplication and manual counting
- Better continuity after application restarts
- A foundation for future real email/SMS, storage, analytics, and online deployment

Benefits not yet proven

The application supports these expected benefits, but the repository contains no completed field evaluation establishing exact time saved, percentage reduction in paper use, adoption rate, error reduction, or user-satisfaction score. These should be measured during usability testing and system evaluation.

4. System Scope and Implementation Status

Capability	Status	Evidence and boundary
Resident registration and login	Implemented	<code>/register</code> , <code>/login</code> ; immediate active Resident account
Administrator-created Staff/Admin users	Implemented	<code>/users</code> , Admin-only POST logic
Admin approval before Resident activation	Planned / Not Yet Implemented	Self-registration creates an active account immediately
Permit workflow	Implemented	Submission, review, endorsement, final decision, pickup, completion
Event workflow	Implemented	Submission through completion
Blotter workflow	Implemented	Filing through closure
Dedicated schedules	Implemented	Related Permit/Event/Blotter appointment records
In-app notifications	Implemented	Persisted and mark-as-read operations
External email and SMS	Partially Implemented	Records and contacts exist; delivery adapter is simulated
Submission receipt PDF	Implemented	Permit, Event, and Blotter acknowledgments
Approved Permit PDF	Implemented	Only for Approved/Ready for Pickup/Completed
Approved Event document	Planned / Not Yet Implemented	No corresponding generator/route
Reports and CSV/PDF export	Implemented	Counts grouped by status/channel
Charts, trends, and date filters	Planned / Not Yet Implemented	Current report metrics have no chart or date-filter logic
Activity logging	Implemented	Central function and Administrator page
Announcements	Implemented	Admin create/delete; authenticated users view published records
Online payment	Planned / Not Yet Implemented	No payment model, route, or dependency
Permanent cloud file storage	Planned / Not Yet Implemented	Uploads currently use local filesystem
General transaction deletion	Planned / Not Yet Implemented	No Permit/Event/Blotter delete endpoints

5. Input-Process-Output Model

Level 1 — Simple explanation

IPO means Input, Process, and Output. It is like a kitchen: ingredients enter, the cook follows steps, and a finished meal comes out. The system receives information, checks and processes it, saves it, and returns useful results.

Inputs

- Resident identity, contact, email, and address
- Username and password
- Permit type, purpose, and attachment
- Event name, type, description, date, location, attendance, and attachment
- Blotter respondent, incident type/date/location, narrative, and evidence
- Schedule type, related record, date/time, and purpose
- Status decision and remarks
- Notification title, message, recipient, and channel
- Announcement and configuration values

Processes

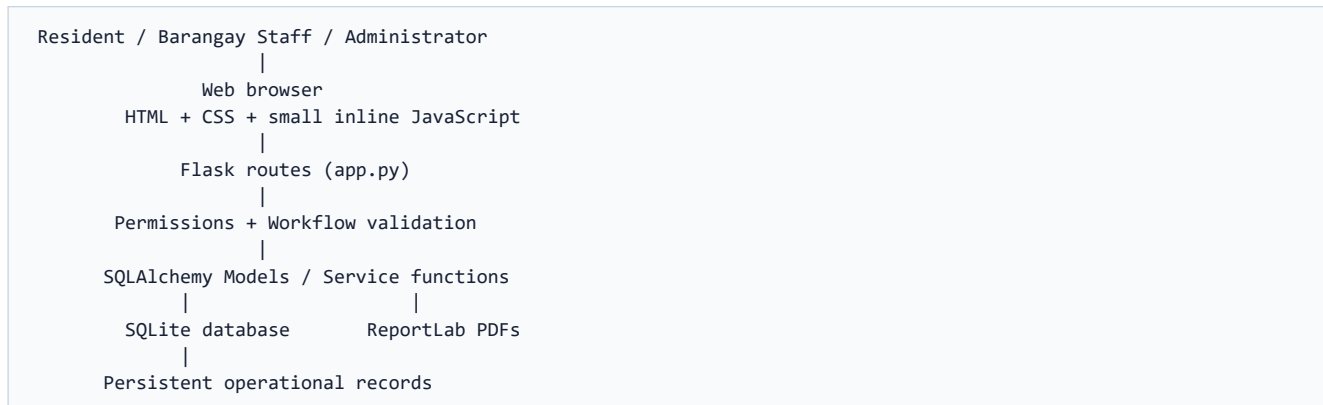
- 1 Authenticate the user and load the role.
- 2 Validate CSRF, required fields, data types, and uploaded extensions.
- 3 Check ownership and role permission.
- 4 Generate a reference number.
- 5 Insert or update SQLAlchemy model objects.
- 6 Create notifications and activity logs.
- 7 Commit the transaction or roll back on handled failure.
- 8 Query SQLite for dashboards, details, reports, and PDFs.

Outputs

- Account and authenticated session
- Permit, Event, Blotter, Schedule, Notification, Announcement, and Log records
- Status and dashboard information
- Submission acknowledgment PDF
- Approved Permit PDF
- CSV/PDF monitoring report
- Error or permission response when a rule is violated

6. System Architecture and Data Flow

Architecture



Level 1 Data Flow Diagram explanation

Process	Information entering	Processing	Data store/output	Roles
1.0 User Management	Registration/profile/user fields	Validate, hash password, assign role	users , activity_logs	Resident, Staff read-only, Admin
2.0 Permit Processing	Type, purpose, attachment, status	Validate, reference, review, decide	permit_applications , permit_types	All roles by permission
3.0 Event Approval	Event details and status	Validate, review, endorse, decide	event_requests , event_categories	All roles by permission
4.0 Blotter Management	Complaint and incident details	File, investigate, schedule, resolve	blotter_cases	All roles by permission
5.0 Scheduling/Notifications	Related record, time, message	Create/update appointment and notices	schedules , notifications	All authorized roles
6.0 Reports/Monitoring	Stored transaction states	Group and count	reports , CSV/PDF	Admin
7.0 Activity Logging	Actor/action/context	Record audit event	activity_logs	System writes; Admin reads

The Resident sends applications and receives status information. Staff perform operational review. The Administrator manages authority, final decisions, configuration, oversight, and reports. The SQLite file on persistent storage is the single source of truth for this one-instance deployment.

7. Users, Roles, and Use-Case Flow

Resident

The Resident can self-register, log in, manage a profile, create Permit/Event/Blotter submissions, request schedules connected to owned records, view status, read announcements and personal notifications, and download authorized receipts/documents. `scoped()` and `ensure_ownership()` prevent access to another Resident's transaction.

Barangay Staff

Staff represent the operational front desk. They can view users, review and endorse Permit/Event requests, process Blotter cases, create/manage schedules, and view announcements. Staff cannot make final Permit/Event approvals, manage roles, view Administrator reports, or open Activity Logs.

Administrator

The Administrator has wildcard permission in `ROLE_PERMISSIONS`. The role can manage users, roles, account activation, announcements, all service records, schedules, notifications, reports, logs, and configuration. Code prevents an Administrator from deactivating their own account or removing their own Administrator role.

Actual permission matrix

Feature	Resident	Staff	Administrator
Self-register	Yes	No separate self-registration role	No separate self-registration role
Login/logout/profile	Own	Own	Own
View announcements	Yes	Yes	Yes
Manage announcements	No	No	Yes
Submit Permit/Event	Own	No	Admin permission is broad, but create UI is Resident-oriented
Review Permit/Event	No	Review and endorse	Full workflow
Final Permit/Event decision	No	No	Yes
File Blotter	Own	Allowed	Allowed by wildcard
Process Blotter	No	Operational stages	Full stages
Schedule	Own related transaction	Operational	Full
Notifications	Own	Limited by permission model	Full/send
User list	No	Read-only	Full management
Reports/export	No	No	Yes
Activity Logs	No	No	Yes
Configuration	No	No	Yes

Use-case sequence

```

Resident: Register -> Login -> Submit -> Receive reference/receipt
          -> View status/notification -> Schedule -> Logout

Staff:    Login -> Review queue -> Under Review -> Endorse/Process
          -> Manage schedule -> Logout

Admin:    Login -> Manage users/configuration -> Final decisions
          -> Notifications/reports/logs -> Logout
  
```

8. Authentication, Registration, and Password Recovery

Login

Simple: The system compares the entered secret with a protected version in the database. If it matches and the account is active, the user receives a session.

Technical flow: `/login` searches `User.username` or `User.email`, calls `check_password()`, checks `is_active`, clears any old session, sets `session.permanent`, stores `user_id`, rotates the CSRF token, writes a `LOGIN` activity record, and commits.

The project does not use Flask-Login. `load_user_and_csrf()` loads the current `User` into Flask's `g` object on every request, and the custom `login_required` decorator protects private routes.

Registration

`/register` requires username, valid-looking email, full name, and a password of at least 10 characters containing a letter and number. Duplicate username/email is checked before insertion. `User.set_password()` stores a Werkzeug hash. The role is forced to `resident`.

Implemented behavior: A valid Resident registration is active immediately. Administrator approval of new Resident accounts is not implemented.

Authentication versus authorization

- Authentication: "Who are you?" Login proves identity.
- Authorization: "What may you do?" Permissions, role rules, and ownership checks decide access.

Forgot Password

```
Enter email -> generic response -> random token issued
-> SHA-256 token hash saved -> simulated email notification recorded
-> token link opened -> expiry/use checked -> new password hashed
-> all unused tokens invalidated -> audit record committed
```

The generic response does not reveal whether an email exists. The raw token is not saved.

`PasswordResetToken.valid_for()` compares the hash and requires `used_at` `IS NULL` and a future `expires_at`. In debug mode only, the development reset link can be displayed; production does not expose it.

9. Role-Specific Dashboards

Administrator Dashboard

The Administrator sees live counts for active Residents, Permit applications, Event requests, open Blotter cases, today's schedules, and Permit/Event records endorsed for final decision. A priority table links endorsed records to review pages. Values come from SQLAlchemy `count()` and filtered queries in `dashboard()`.

Staff Dashboard

Staff see requests awaiting review, Under Review totals, new Filed Blotters, items endorsed to Admin, and today's appointments. The operational table links pending Permit/Event records to review pages. It is a work queue, not a management analytics dashboard.

Resident Dashboard

Residents see active applications, total personal Blotter reports, upcoming personal appointments, unread in-app notifications, quick service actions, recent Permit/Event records, receipt links, the next schedule, and recent notices. Queries use the signed-in user ID.

Boundary: The current dashboard does not render graphical charts. The report metrics are counts and grouped summaries.

10. Module 1.0 — User Management

Simple explanation

The Users table is the system's list of people and their keys. A role determines which doors each person can open.

Technical explanation

User stores `id`, unique `username`, `password_hash`, `role`, `full_name`, contact details, unique email, address, active state, and timestamps. Residents self-register. An authenticated Administrator can create Resident, Staff, or Admin accounts, change roles, and activate/deactivate another account. Staff can only view the user list.

Important protections include password hashing, backend role validation, unique database constraints, prevention of self-deactivation, prevention of removing one's own Admin role, CSRF protection, and audit entries.

Current limitations

- No email verification.
- No multi-factor authentication.
- No Resident approval queue.
- User creation does not provide a friendly duplicate-integrity handler in every path.
- No user deletion route, which also helps preserve historical relationships.

11. Module 2.0 — Permit Processing

Workflow

```
Resident submits  
  -> Pending  
Staff/Admin -> Under Review  
Staff/Admin -> Endorsed to Admin  
Admin -> Approved or Rejected  
Approved -> Ready for Pickup  
Ready for Pickup -> Completed
```

The Administrator can also cancel a Pending Permit. Staff cannot approve, reject, cancel, mark ready, or complete a Permit.

Stored information

The `permit_applications` table stores the reference, applicant, Permit type, purpose, status, dates, reviewers/decision users, remarks, approved-file path, attachment path, and timestamps. `permit_types` supplies name, description, requirements, fee, validity, and active state.

Submission and review

`POST /permits` verifies the `permit:create` permission, validates Permit type and purpose, checks the optional file extension, creates the row, flushes to obtain ID/reference, audits the submission, creates notifications, and commits. `/permits/<id>` applies ownership for Residents and uses `update_status()` for authorized workflow changes.

Approved document

`/permits/<id>/approved-document` is allowed only when status is Approved, Ready for Pickup, or Completed. ReportLab generates a temporary official-looking Permit PDF from the saved application, Resident, and Permit type. The temporary file is deleted after the response.

12. Module 3.0 — Event Approval Management

Workflow

```
Pending -> Under Review -> Endorsed to Admin  
                -> Approved -> Completed  
                -> Rejected
```

The Administrator may cancel a Pending Event. Staff review and endorse but cannot make the final decision.

Stored fields

The `event_requests` model includes reference, requester, event name, category, type, description, proposed date, start/end times, location, attendees, supporting document, status, reviewer, decision date, remarks, and timestamps.

Actual current form behavior

The route currently saves event name, type, description, proposed date, location, expected attendees, and attachment. At least one attendee is required. Although `category_id`, `proposed_start_time`, and `proposed_end_time` exist in the model, the current submission route does not populate them. These fields are therefore Partially Implemented.

13. Module 4.0 — Blotter Management

Simple explanation

A Barangay Blotter is a formal record that a complaint or incident was reported. It is not automatically a court judgment; it records the matter and supports barangay processing.

Workflow

```
Filed -> Under Review -> For Investigation  
      -> For Hearing / Scheduled -> Resolved -> Closed
```

The Administrator may resolve directly from For Investigation. Staff can move records through the hearing stage but cannot resolve or close them.

Data and privacy

The table records complainant, respondent, incident type/date/location, narrative, supporting file, status, filed date, officer, resolution, hearing date, updater, and timestamps. Residents can access only their own cases; Staff and Administrators have wider operational access. Because narratives may contain sensitive personal allegations, production deployment should use TLS, strict accounts, backups, least privilege, secure storage, retention rules, and organizational privacy procedures.

Partial fields

The current submission route saves respondent name, incident type/date/location, narrative, and attachment. `respondent_address`, `assigned_officer`, `resolution`, `hearing_date`, and `updated_by` exist in the model but are not fully populated by the present generic status form.

14. Module 5.0 — Scheduling and Notifications

Scheduling

A Schedule row is like an appointment card pointing to its Permit, Event, or Blotter. `related_type` says which kind of record; `related_id` gives that record's ID. `related_owner()` verifies that the record exists and finds its owner.

Schedule statuses are Requested, Confirmed, Rescheduled, Completed, and Cancelled. Residents can create appointments only for their own transactions. Staff and Admin may create operational schedules; only non-Residents can change schedule status.

Database design note: `related_id` is polymorphic and has no direct foreign key to three different tables. Application code enforces existence, but the database cannot enforce that relationship by itself.

Notifications

The Notification model stores recipient IDs/contact, title, message, channel, delivery status, related record, sent/read timestamps, and creation date. In-app messages can be read individually or all at once. The global template context counts unread in-app records.

`notify_status()` creates three records: in-app, email, and SMS. In-app is marked sent. Email/SMS are marked sent only when a contact exists, but the code comment explicitly says external delivery is simulated.

Announcements

The Administrator can publish or delete announcements. Authenticated Staff and Residents see only published, non-expired records. The model supports expiration, but the creation form currently supplies title/body only.

15. Submission Receipts and Approved Documents

Document	Meaning	When available	Data source
Submission acknowledgment	Proof that the system received a request	Immediately after committed Permit/Event/Blotter submission	Saved operational record
Approved Permit document	Evidence of an authorized Permit state	Approved, Ready for Pickup, or Completed	Permit, applicant, Permit type

The acknowledgment includes the reference, submission type, Resident, date/time, current received state, service details, requirements guidance, processing estimate, next steps, and a clear non-approval disclaimer.

`receipt_record()` enforces ownership for Residents. The receipt can be regenerated because it is built from the database, not a one-time browser value.

Receipt generation happens after the submission transaction exists. If generation later fails, the committed request is not deleted. The route logs the exception, rolls back the PDF audit transaction, writes a failure audit event, and returns a 500 page.

16. Module 6.0 — Reports and Monitoring

`report_metrics()` performs grouped counts:

- Permits by status
- Events by status
- Blotters by status
- Schedules by status
- Notifications by channel

Only the Administrator has `reports:view` and `reports:export`. Export creates a `reports` history row and audit event. CSV contains module/status/count rows. PDF uses ReportLab to render the same summary.

Implemented: database-derived grouped totals, CSV export, PDF export, export history. Planned / Not Yet

Implemented: graphical charts, trend calculations, date range filters, status filter controls, scheduled reports, and advanced analytics.

17. Module 7.0 — Activity Logging

Simple explanation

The Activity Log is a digital security notebook. It remembers important actions such as login, submission, status change, receipt generation, user changes, and report export.

Technical explanation

`log_activity()` accepts `user_id`, action, details, IP, resource, target reference, role, user agent, and a commit choice. When possible, it derives the actor role from the Users table. Most routes call the `audit()` wrapper with `commit=False`, allowing the business change and its audit record to commit together.

The Administrator route `/activity-logs` shows the most recent 500 events. There is no edit/delete UI, helping preserve audit history, although database administrators still require procedural controls.

18. SQLite and Complete Database Architecture

SQLite in simple words

SQLite is the system's digital storage room contained in one database file. Tables are labeled cabinets, columns are labeled spaces, and rows are individual records. Closing Flask does not erase committed rows. On Render, this remains true only when the database file is under the attached Persistent Disk at `/var/data`; the normal Render filesystem is temporary.

Connection architecture

`config.py` reads `DATA_DIR` and `SQLITE_DB_PATH` from a private `.env` or production environment. It builds an absolute URL similar to:

```
sqlite:///var/data/minante.db
```

SQLite has no separate database username/password. The application still requires a strong private `SECRET_KEY` and protected Administrator credentials. Each SQLite connection receives `foreign_keys=ON`, a 30-second busy timeout, WAL journaling, and `synchronous=NORMAL`. These settings enforce declared relationships and reduce transient lock failures for the intended single-instance workload.

SQLAlchemy versus raw SQL

Normal application CRUD uses SQLAlchemy ORM. Migration code uses controlled raw SQL through SQLAlchemy `text()` for additive `ALTER TABLE` statements and legacy status normalization. User form values are not concatenated into SQL. The normal application uses Python's built-in `sqlite3` driver through SQLAlchemy; PyMySQL is isolated to the optional one-time MariaDB-to-SQLite migration dependency.

Complete table reference

Table	Purpose and important columns	Keys and relationships	Main operations
<code>users</code>	Identity, hash, role, name, contact, email, address, active state	PK <code>id</code> ; unique username/email	Register, read, profile/role/status update
<code>permit_types</code>	Permit catalog, requirements, fee, validity	PK <code>id</code> ; unique name	Admin create; read by applications/config
<code>event_categories</code>	Event category catalog	PK <code>id</code> ; unique name	Admin create; currently not saved by Event route
<code>permit_applications</code>	Permit request and workflow	FK applicant/type/decision users	Create, read, status update
<code>event_requests</code>	Event request and workflow	FK requester/category/reviewer	Create, read, status update
<code>blotter_cases</code>	Complaint, incident, workflow, resolution fields	FK complainant/updater	Create, read, status update
<code>schedules</code>	Appointment linked by type/ID	FK requester/confirmer	Create, read, status update
<code>notifications</code>	In-app/email/SMS history	FK user/legacy recipient	Create, read, mark read
<code>activity_logs</code>	Audit events	FK user	Create; Admin read
<code>reports</code>	Export history	FK generated_by	Create; indirect read/history
<code>password_reset_tokens</code>	Hashed recovery token, expiry/use	FK user; unique hash	Issue, validate, invalidate
<code>system_settings</code>	Key/value configuration	PK; unique key	Read/update receipt estimates
<code>announcements</code>	Community messages and visibility	FK created_by	Admin create/delete; authenticated read

Primary and foreign keys

A primary key is like a unique student ID. A foreign key is an ID written on another form to show ownership. For example, `users.id` is referenced by `permit_applications.applicant_id`; one User may therefore own many Permit applications.

Simplified relationship diagram

```
users 1 ---- many permit_applications ---- 1 permit_types
users 1 ---- many event_requests ----- 0..1 event_categories
users 1 ---- many blotter_cases
users 1 ---- many schedules
users 1 ---- many notifications
users 1 ---- many activity_logs
users 1 ---- many reports
users 1 ---- many password_reset_tokens
users 1 ---- many announcements

schedules --(related_type + related_id, application-enforced)-->
    permit_applications OR event_requests OR blotter_cases
```

Constraints and indexes

`db.create_all()` creates the current SQLAlchemy schema, including primary/foreign keys, unique references, and model indexes. `migration.py` performs only additive legacy upgrades and status normalization. `seed.py` preserves an existing Administrator and adds legitimate Permit/Event catalogs—never demo Staff or Resident accounts. The old MariaDB SQL files are historical migration references and are not used by the running SQLite system.

Render persistence boundary

The production database and uploads live beneath `/var/data`, the mount path declared in `render.yaml`. SQLite is safe here only with one Render service instance and one Gunicorn worker. A free Render web service cannot attach this disk, so a free deployment would lose new records and uploads on restarts or redeploys. An attached disk also means brief downtime during deploys and prevents horizontal scaling. `backup_sqlite.py` creates a consistent verified backup; Render disk snapshots provide an additional recovery layer.

19. CRUD, Transactions, ORM, and Reference Numbers

CRUD

- Create: Resident submits a Permit; SQLAlchemy inserts a row.
- Read: Administrator opens a Permit or grouped report.
- Update: Staff changes Pending to Under Review.
- Delete: Administrator deletes an Announcement. General transaction deletion is not implemented.

Commit and rollback

`commit()` means “save this complete unit permanently.” `rollback()` means “discard unfinished database changes after a failure.” Submission, notification, and audit records are generally committed together. Some routes explicitly roll back validation/PDF errors, and the 500 handler rolls back the session.

References

`make_reference(prefix)` returns a prefix, UTC year, and six-digit cryptographically generated random value, for example `PRM-2026-123456`, `EVT-2026-123456`, `BLT-2026-123456`, or `SCH-2026-123456`. The model uses `PRM`, not `PMT`. Unique database constraints detect collisions, but the current code does not retry automatically after a rare collision.

20. File Uploads and Storage

`save_upload()` accepts PDF, PNG, JPG, and JPEG. It checks the final extension, sanitizes the original name with `secure_filename()`, prefixes 16 random bytes in hexadecimal, creates the upload folder, and saves the file. Flask's configured maximum request content is 8 MB.

Implemented protections: extension allowlist, randomized name, filename sanitization, size ceiling, database path association, authenticated route entry. Limitations: no content-signature/MIME inspection, malware scan, image re-encoding, object storage, retention job, or dedicated authorized download route for every attachment. Local deployment storage may be ephemeral on a cloud host.

21. Role-Based Access Control and Status Transitions

RBAC analogy

RBAC is like issuing different keys. Residents receive keys to their own services. Staff receive front-desk keys. Administrators receive management keys.

Enforcement layers

- 1 `login_required` checks authentication and active state.
- 2 `permission_required()` checks centralized grants.
- 3 `can()` supports route and template decisions.
- 4 `scoped()` limits Resident lists.
- 5 `ensure_ownership()` prevents opening another Resident's record.
- 6 `validate_transition()` blocks invalid workflow jumps.
- 7 CSRF checks protect every POST/PUT/PATCH/DELETE request.

Hiding a button is not security; backend checks are the actual protection. Unauthorized access produces 401/redirect or a 403 page.

22. Frontend Architecture and User Experience

`auth_base.html` provides the split authentication layout. `base.html` provides the role-aware sidebar, header, notification badge, profile menu, flash messages, responsive mobile drawer, skip link, and main content area. Other templates extend one of these foundations.

The UI uses Jinja2, semantic HTML, custom CSS, and small inline JavaScript. JavaScript manages the sidebar/drawer, Escape-key closing, remembered collapsed state, and button feedback during form submission. There is no Bootstrap dependency and no separate JavaScript framework.

The base template is like the frame of a house: each page supplies different content while navigation, header, spacing, and behavior remain consistent.

23. Programming Languages, Frameworks, and Libraries

Technology	Type	Actual purpose
Python	Programming language	Routes, validation, workflows, database logic, PDFs
HTML	Markup language	Page and form structure
CSS	Styling language	Responsive navy/gold interface
JavaScript	Programming language	Drawer and form interaction
SQL	Query language	SQLite schema/migrations, pragmas, and generated ORM queries
Flask 3.1.3	Web framework	HTTP application and routing
Jinja2	Template engine (through Flask)	Server-rendered dynamic pages
Flask-SQLAlchemy 3.1.1	ORM integration	Models, queries, sessions, transactions
Werkzeug 3.1.8	Library	Password hashes and safe filenames
ReportLab 5.0.0	PDF library	Receipts, Permits, and reports
python-dotenv 1.1.1	Configuration library	Private environment loading
Gunicorn 26.2.0	Production WSGI server	Runs one process with multiple threads on Render
SQLite / <code>sqlite3</code>	Database technology/driver	Single-file persistent relational data

24. Project Structure

```

barangay_system/
|-- app.py                Flask routes and orchestration
|-- models.py             SQLAlchemy models and audit helper
|-- permissions.py        RBAC and navigation
|-- workflows.py          Status-transition rules
|-- config.py             SQLite paths/environment configuration
|-- migration.py          Additive legacy upgrades
|-- migrate_mariadb_to_sqlite.py Optional one-time legacy transfer
|-- backup_sqlite.py      Consistent integrity-checked backup
|-- seed.py               Admin-safe catalog initialization
|-- requirements.txt       Python dependencies
|-- requirements-migration.txt Optional MariaDB source driver
|-- database/
|   |-- legacy MariaDB SQL references (not operational)
|-- instance/minante.db    Local SQLite file (git-ignored)
|-- instance/uploads/      Local persistent uploads (git-ignored)
|-- services/
|   |-- notify.py          Notification records/adapters
|   |-- pdfgen.py          PDF generation
|-- templates/             Jinja2 pages and shared layouts
|-- static/css/style.css   Visual system
|-- tests/
|   |-- test_system.py     Functional/RBAC tests
|   |-- validate_sqlite.py SQLite integration/persistence validation
|   |-- sqlite_restart_probe.py Separate-process persistence probe
|-- README.md
|-- USE_CASE_MATRIX.md
`-- UI_UX_REVIEW.md

```

The repository root also contains `render.yaml`, which declares the Render web service, `/var/data` persistent disk, health check, environment values, and single-worker Gunicorn command.

The code is currently organized as a compact Flask application rather than separate Blueprints/packages. This is valid for its present size but may be modularized as the project grows.

25. Important Code Explanations

Database URL

```
SQLITE_DB_PATH = "/var/data/minante.db"
SQLALCHEMY_DATABASE_URI = "sqlite:///var/data/minante.db"
```

This selects SQLAlchemy's SQLite dialect and the database file on Render's mounted disk. Locally the configured absolute path points to `instance/minante.db` instead.

SQLite connection protection

```
cursor.execute("PRAGMA foreign_keys=ON")
cursor.execute("PRAGMA busy_timeout=30000")
cursor.execute("PRAGMA journal_mode=WAL")
cursor.execute("PRAGMA synchronous=NORMAL")
```

Foreign-key enforcement is enabled on every connection. The busy timeout waits for a short-lived writer rather than failing immediately, and WAL improves read/write coexistence. SQLite still has one writer at a time, which is why Render uses one application instance and one Gunicorn worker.

Password protection

```
def set_password(self, raw):
    self.password_hash = generate_password_hash(raw)
```

Input is a raw password; processing is a salted one-way hash; output is a stored hash. The original password is not recoverable or displayed.

Reference generation

```
def make_reference(prefix):
    return f"{prefix}-{utcnow().year}-{secrets.randbelow(1000000):06d}"
```

The function combines module prefix, UTC year, and a zero-padded secure random number. A database unique constraint is the final uniqueness guard.

Ownership

```
def ensure_ownership(record, owner_id):
    if actor().role == "resident" and owner_id != actor().id:
        abort(403)
```

Residents must own the record. Staff/Admin are governed by their wider role rules. The unused `record` argument could be removed in a future refactor.

Status validation

```
if not validate_transition(kind, actor().role, old, target):
    abort(400, "Invalid status transition.")
```

The server checks resource, role, current state, and requested target. A forged form cannot bypass the state machine.

Audit in the same transaction

```
audit("PERMIT_SUBMITTED", "permit", item.reference_no)
notify_status(...)
db.session.commit()
```

The application, audit, and notification work are intended to be committed as one logical change. A failure before commit prevents a partially saved ORM transaction.

Secure reset token storage

```
token_hash = hashlib.sha256(raw.encode()).hexdigest()
```

The browser/email receives the raw token, while SQLite receives only its hash. Expiry and used timestamps prevent indefinite/repeated use.

26. Route and Request Flow

Route	Methods	Purpose	Main roles/data
/login	GET/POST	Authenticate	All; users , activity_logs
/register	GET/POST	Resident account	Public; users
/forgot-password	GET/POST	Request recovery	Public; tokens/notifications/logs
/reset-password/<token>	GET/POST	Change password	Token holder
/dashboard	GET	Role-specific overview	Authenticated; multiple tables
/profile	GET/POST	Profile/password	Authenticated owner
/announcements	GET/POST	View/Admin publish	Authenticated/Admin
/permits	GET/POST	List/submit	All scoped; Resident create
/permits/<id>	GET/POST	Detail/workflow	Owner/Staff/Admin
/events	GET/POST	List/submit	All scoped; Resident create
/events/<id>	GET/POST	Detail/workflow	Owner/Staff/Admin
/blotters	GET/POST	List/file	Authorized roles
/blotters/<id>	GET/POST	Detail/workflow	Owner/Staff/Admin
/schedules	GET/POST	List/create	Authenticated by permissions
/schedules/<id>/status	POST	Status change	Staff/Admin
/notifications	GET	Personal inbox	Authenticated
/notifications/send	POST	Create notice	Admin under current permission map
/users	GET/POST	View/Admin create	Staff read; Admin manage
/reports	GET	Monitoring	Admin
/reports/export/<fmt>	POST	CSV/PDF	Admin
/activity-logs	GET	Audit history	Admin
/configuration	GET/POST	Catalog/settings	Admin

GET normally reads or displays. POST changes server state and requires CSRF. The current application does not expose a JSON REST API, PUT/PATCH endpoints, or general DELETE HTTP routes; Announcement deletion is a POST action.

27. End-to-End Use Cases

Example 1 — Permit

Maria logs in, opens `/permits`, selects Barangay Clearance, gives a purpose, and optionally uploads a permitted file. Flask checks her session, CSRF token, permission, Permit type, purpose, and extension. SQLAlchemy creates `permit_applications`, `notifications`, and `activity_logs` records and commits. Maria receives a `PRM` reference and downloads an acknowledgment. Staff changes Pending to Under Review and then Endorsed to Admin. The Administrator approves it, later marks it Ready for Pickup, and finally Completed. An approved Permit PDF is available after approval.

Example 2 — Event

Maria submits a valid event name/type/description/date/location and attendee count. The system creates an `EVT` reference, notifications, audit entry, and receipt. Staff reviews and endorses. The Administrator approves/rejects and can complete an approved event record. No approved Event PDF is currently generated.

Example 3 — Blotter

Maria records respondent name and incident details. The system creates a `BLT` case, audit entry, notices, and receipt. Staff performs review, investigation, and hearing-stage processing. The Administrator may resolve and close the case. The record remains restricted from other Residents.

28. Security and Error Handling

Implemented protections

- Werkzeug password hashing
- Hashed, expiring, one-use reset tokens
- Custom authentication and active-account check
- Central RBAC
- Resident ownership filtering
- CSRF token validation for state changes
- SQLAlchemy-bound parameters
- Restricted upload extensions, sanitized/random names, and 8 MB request ceiling
- HttpOnly and SameSite=Lax session cookie settings
- Activity logging
- Generic password-recovery response
- 400, 403, 404, and 500 handlers
- Database rollback in the 500 handler and selected operations

HTTP errors

- 400 Bad Request: malformed input, invalid CSRF, missing related record, or illegal transition.
- 401 Unauthorized: unauthenticated permission request; application redirects to login.
- 403 Forbidden: signed-in user lacks role/ownership/status permission.
- 404 Not Found: route or record does not exist.
- 500 Internal Server Error: unexpected server/database/PDF failure; session is rolled back and a generic page is shown.

Production hardening still required

Set a strong `SECRET_KEY`; enable Secure cookies under HTTPS; use a production WSGI server; add connection timeouts/health endpoint; use real external notification adapters; move uploads to private object storage; validate MIME/content; add rate limiting and login throttling; centralize structured logs; use versioned migrations; test backup restoration; and avoid running schema upgrades implicitly during every process import.

29. From Click to Database: What Happens Behind the Screen

- 1 The Resident clicks Submit.
- 2 The browser sends form fields, optional file, session cookie, and CSRF token.
- 3 Flask selects the matching POST route.
- 4 `before_request` loads the user and validates CSRF.
- 5 The route checks `can()` and/or ownership.
- 6 Required values, dates, counts, and extension are validated.
- 7 The upload is sanitized and saved if present.
- 8 SQLAlchemy creates the Permit model object.
- 9 `flush()` obtains the new ID and generated reference without finalizing yet.
- 10 An Activity Log and three notification-channel records are added.
- 11 `commit()` permanently saves the transaction to the SQLite file.
- 12 Flask redirects to the success route.
- 13 The success page displays the reference and receipt link.
- 14 When requested, ReportLab re-reads the saved data and returns a PDF.

This sequence shows why database commit occurs before receipt generation: proof should be generated only for a real saved record.

30. Feature-to-Code-to-Database Matrix

Feature	Main code	Main tables	Users
Login/session	<code>login, load_user_and_csrf</code>	users, activity_logs	All
Registration	<code>register, User.set_password</code>	users, activity_logs	Resident
Password reset	<code>PasswordResetToken.issue/valid_for</code>	password_reset_tokens, users	All
Permit	<code>permits, permit_detail, update_status</code>	permit_applications, permit_types	All by role
Event	<code>events, event_detail, update_status</code>	event_requests, event_categories	All by role
Blotter	<code>blotters, blotter_detail</code>	blotter_cases	All by role
Scheduling	<code>schedules, schedule_status</code>	schedules	All authorized
Notifications	<code>create_notification, notify_status</code>	notifications	All
Announcements	<code>announcements</code>	announcements	All/Admin manage
Receipts	<code>submission_receipt, generate_submission_receipt</code>	Permit/Event/Blotter data	Authorized
Approved Permit	<code>approved_permit_document</code>	permit_applications/users/types	Authorized
Reports	<code>report_metrics, report_export</code>	multiple + reports	Admin
Audit	<code>audit, log_activity</code>	activity_logs	System/Admin read
Configuration	<code>configuration</code>	permit_types, event_categories, system_settings	Admin

31. Strengths, Limitations, and Future Improvements

Confirmed strengths

- Real relational persistence through SQLite on a Render Persistent Disk
- Three-role separation and backend enforcement
- Owner-scoped Resident access
- Dedicated Permit, Event, Blotter, Schedule, Notification, Report, and Audit stores
- Explicit workflow state machines
- Submission proof separate from approval
- Hashed passwords and reset tokens
- CSRF validation and parameterized ORM queries
- Administrator reports and audit trail
- Responsive, consistent UI
- Production-safe seed behavior with no demo Staff/Resident accounts

Confirmed limitations

- External email/SMS is simulated.
- SQLite permits one writer at a time and this deployment cannot scale to multiple instances.
- A Render Persistent Disk requires a paid web service and disables zero-downtime deploys.
- Uploads remain filesystem-based and must stay beneath `/var/data` to persist.
- No multi-factor authentication, rate limiting, or email verification.
- Reports have no charts, trends, or date/status filter UI.
- Some modeled Event and Blotter fields are not populated by current forms.
- No approved Event PDF or general transaction deletion workflow.
- No online payment integration.
- Reference collision has a unique constraint but no automatic retry.
- Polymorphic Schedule relationships are application-enforced, not foreign-key-enforced.
- SQLite backup/download and restore procedures must be operated and tested regularly.

Recommended future work

- 1 Keep Flask behind Render HTTPS with the declared single-worker Gunicorn command.
- 2 Monitor the persistent disk, retain verified SQLite backups, and rehearse restore procedures.
- 3 Move uploads/PDF artifacts to private object storage if storage or privacy needs grow.
- 4 Introduce Alembic/Flask-Migrate and backward-compatible SQLite batch migrations.
- 5 Add monitoring, alerts, and tested application/disk restores.
- 6 Integrate a real Philippine-compatible SMS/email provider.
- 7 Add rate limiting, MFA for privileged accounts, and verified email.
- 8 Complete Event category/time and Blotter officer/resolution forms.

- 9 Add accessible charts and real date/status report filters.
- 10 Conduct usability, performance, security, and community-impact evaluation.

32. Common Research and Defense Questions

- 1 What is the system's main purpose? To integrate Permit, Event, Blotter, Schedule, Notification, Report, and audit workflows for Barangay Minante 1.
- 2 What problem does it solve? It addresses fragmented records, unclear status, repeated follow-ups, inconsistent workflows, scheduling gaps, and limited oversight.
- 3 Who are its users? Residents, Barangay Staff, and Administrators.
- 4 What database does it use? SQLite through Flask-SQLAlchemy and Python's built-in `sqlite3` driver.
- 5 Why SQLite? It keeps the database inside the same Render service, requires no separately deployed database, and still provides relational tables, transactions, constraints, and indexes for a modest single-instance workload.
- 6 Does restarting Flask erase records? No when the file is stored at `/var/data/minante.db` on the attached Persistent Disk. A free/ephemeral Render deployment would erase it.
- 7 What is Flask? The Python framework that receives requests, runs logic, and returns pages/files.
- 8 What is SQLAlchemy? The ORM used to represent tables as Python classes and generate parameterized SQL.
- 9 Does the system use Flask-Login? No. It implements custom session authentication.
- 10 What is authentication? Proving who a user is.
- 11 What is authorization? Deciding what an authenticated user may do.
- 12 How are passwords protected? Werkzeug stores one-way password hashes.
- 13 Can the old password be displayed? No; a secure hash is not reversible.
- 14 How does Forgot Password work? A random, expiring, one-use token is issued and only its SHA-256 hash is stored.
- 15 Why use a generic recovery response? To prevent attackers from discovering registered email addresses.
- 16 Does registration require Admin approval? No; current valid Resident registration is active immediately.
- 17 Can Staff approve a Permit? No; Staff can review and endorse, while Admin makes final decisions.
- 18 What is RBAC? Role-Based Access Control, which grants permissions according to job role.
- 19 How is Resident privacy enforced? List queries are scoped by user ID and detail routes check ownership.
- 20 Why are backend checks important? Hidden buttons can be bypassed; server checks enforce real security.
- 21 What is CSRF? A forged-request attack; the system requires a session token for state-changing requests.
- 22 What is a primary key? A unique identifier for one table row.
- 23 What is a foreign key? A column that connects a row to another table's primary key.
- 24 Give a relationship example. `permit_applications.applicant_id` points to `users.id`.
- 25 What is CRUD? Create, Read, Update, and Delete.
- 26 Does every module support deletion? No; general Permit/Event/Blotter deletion is not implemented.
- 27 What does commit do? Permanently completes the current database transaction.
- 28 What does rollback do? Cancels unfinished changes after a failure.
- 29 What is a transaction? A group of database operations treated as one unit.
- 30 Why generate a reference number? It gives the user and office a concise tracking identifier.
- 31 What reference prefixes are used? `PRM`, `EVT`, `BLT`, and `SCH`.
- 32 How are collisions prevented? Unique constraints reject duplicate reference values.

- 33 What happens after a Permit submission? The record, audit event, and notifications are committed, then the success page offers a receipt.
- 34 Is the receipt a Permit approval? No. It proves receipt only and contains a disclaimer.
- 35 When is an approved Permit PDF available? At Approved, Ready for Pickup, or Completed status.
- 36 Does Event approval generate an official PDF? No; that feature is not implemented.
- 37 Why is Schedule a separate table? It can represent appointments consistently across several transaction types.
- 38 What do `related_type` and `related_id` mean? They identify the kind and ID of the related transaction.
- 39 Are Schedule relationships foreign keys? User links are; the polymorphic related record is checked by application logic.
- 40 Are SMS and email real? No; current external delivery is simulated.
- 41 What notification channel fully works internally? In-app notification storage, display, unread count, and read state.
- 42 How are reports calculated? SQLAlchemy groups records by status/channel and counts them.
- 43 Are charts implemented? No; current output is grouped metrics and CSV/PDF tables.
- 44 What is Activity Logging? A database audit history of important system/security actions.
- 45 Who can view Activity Logs? The Administrator.
- 46 Which uploads are accepted? PDF, PNG, JPG, and JPEG, within the Flask request-size limit.
- 47 How are filenames protected? They are sanitized and prefixed with random hexadecimal text.
- 48 What upload security remains needed? MIME/signature validation, malware scanning, private object storage, and retention controls.
- 49 What happens on unauthorized access? The server redirects unauthenticated users or returns a 403 page.
- 50 What happens on an unexpected failure? The 500 handler rolls back the database session and displays a generic message.
- 51 Why separate Staff and Admin? Separation of duties reduces unauthorized final decisions.
- 52 Where do dashboard values come from? Live SQLite queries in the `dashboard()` route.
- 53 Are dashboard values hardcoded? The labels are coded, but operational counts and records come from database queries.
- 54 What languages are used? Python, HTML, CSS, JavaScript, and SQL.
- 55 What major libraries are used? Flask, Flask-SQLAlchemy, Werkzeug, Gunicorn, ReportLab, python-dotenv, and Python's built-in `sqlite3` driver.
- 56 What is Jinja2? The server-side template engine Flask uses to place real data in HTML.
- 57 Why use environment variables? They keep the secret key, initial Administrator credentials, security mode, and storage paths out of source code.
- 58 What is the permanent Administrator rule? Initialization preserves an existing Admin and never creates demo Staff/Resident accounts.
- 59 What is the biggest deployment risk today? Mounting SQLite or uploads outside `/var/data`, using the free tier, or scaling beyond one instance would cause data loss or fragmentation.
- 60 How should project benefits be evaluated? Measure task completion time, errors, follow-up visits, satisfaction, uptime, and record accuracy before and after adoption.

33. Glossary

Term	Easy meaning
API	A defined way for software components to communicate; no public JSON API is currently exposed
Authentication	Checking who the user is
Authorization	Checking what the user may do
Backend	Flask code that runs rules and database operations
CSRF	An attack that tricks a signed-in browser into sending an unwanted request
CRUD	Create, Read, Update, Delete
Database	Organized persistent digital storage
DFD	Diagram showing how information moves between users, processes, and stores
Foreign Key	A field linking one table to another
Framework	Reusable foundation for building software, such as Flask
Frontend	Pages and controls users see in the browser
HTTP	Protocol browsers and web servers use
Index	Database structure that speeds common searches
Persistent Disk	Render storage mounted at <code>/var/data</code> that survives restarts and redeploys
SQLite	The single-file relational database used by the system
Migration	Controlled change or transfer of database structure/data
Model	Python class describing a database table
ORM	Tool that maps Python objects to database rows
Primary Key	Unique row identifier
RBAC	Access permissions based on role
Reference Number	Human-friendly transaction tracking identifier
Rollback	Cancel unfinished transaction changes
Route	URL and function handling a web request
Session	Signed browser state identifying the logged-in user
SQL	Language used to work with relational databases
Token	Random secret value used for CSRF or password recovery
Transaction	Database changes completed as one unit
UI	User Interface
UX	User Experience
Validation	Checking that input and actions follow rules

34. Quick Reviewer / Cheat Sheet

Purpose: Integrate Barangay Minante 1 Permit, Event, Blotter, Schedule, Notification, Report, and audit services. Users: Resident, Barangay Staff, Administrator. Seven processes: Users; Permits; Events; Blotters; Scheduling/Notifications; Reports; Activity Logs. Backend: Python 3 and Flask. Database: SQLite through Flask-SQLAlchemy; `/var/data/minante.db` on Render. Frontend: Jinja2 HTML, custom CSS, small inline JavaScript. PDF: ReportLab. Authentication: Custom Flask session; Werkzeug password hash. Authorization: `permissions.py`, ownership checks, `workflows.py`. Main tables: 13 tables listed in Chapter 18. Permit path: Pending → Under Review → Endorsed → Approved/Rejected → Pickup → Completed. Event path: Pending → Under Review → Endorsed → Approved/Rejected → Completed. Blotter path: Filed → Review → Investigation → Hearing → Resolved → Closed. Proof of submission: Regenerable PDF acknowledgment, not approval. External SMS/email: Simulated. Main problems solved: fragmented records, unclear status, missing acknowledgment, inconsistent handoffs, scheduling gaps, communication gaps, manual monitoring, and weak accountability. Main benefits: faster retrieval, clearer tracking, controlled decisions, personal status access, persistent records, reports, and audit history.

35. Verification Checklist

- [x] Current Python routes and helper functions inspected
- [x] Models and 13 database tables documented
- [x] SQLite connection, durability pragmas, and Render disk approach documented safely
- [x] Resident, Staff, and Administrator permissions documented
- [x] Permit, Event, Blotter, Schedule, Notification, Report, and Audit workflows documented
- [x] Password hashing and recovery documented
- [x] Submission receipt and approved Permit distinguished
- [x] Actual frontend architecture and technologies documented
- [x] Problem–solution–benefit analysis added
- [x] Implemented, partial, and unimplemented items distinguished
- [x] More than 50 defense questions answered
- [x] Glossary and quick reviewer included
- [x] Sensitive credentials excluded
- [x] Claims not proven by source code qualified honestly

Final Summary

The Barangay Minante 1 Integrated Management System turns several related barangay services into one controlled digital workflow. Its central contribution is not merely replacing paper with a webpage. It connects identity, ownership, review responsibility, final decision authority, scheduling, communication, reporting, and accountability around persistent SQLite records.

For Residents, the system provides easier submission, references, status visibility, schedules, notifications, and proof of receipt. For Staff, it provides a structured operational queue and consistent processing rules. For Administrators, it provides final authority, configuration, live monitoring, reports, and audit visibility. Its current limitations—especially simulated external messaging, single-instance SQLite scaling, filesystem-based uploads, and incomplete advanced reporting—are identifiable and can be addressed without misrepresenting what already works.

The most accurate one-sentence explanation is:

The system is a role-controlled digital barangay service desk that stores real transactions in SQLite on persistent storage, moves them through authorized workflows, and gives users traceable status, schedules, notifications, documents, reports, and accountability.